Đại học Quốc gia Thành phố Hồ Chí Minh
Trường Đại học Công nghệ Thông tin
Khoa Khoa học máy tính

Báo cáo cuối kỳ

Principal Component Analysis (PCA)

Giảng viên hướng dẫn

TS. Nguyễn Vinh Tiệp

Sinh viên thực hiện

Nguyễn Khả Tiến - 19522337
Cát Văn Tài - 19522147
Nguyễn Lê Anh Quân - 19522081

Lớp

CS116.M11.KHCL

Ho Chi Minh City - 2021

Mục lục

1 Giới thiệu đồ án: 3
1.1 Sơ lược về đồ án . . . 3
1.2 Mục tiêu của đồ án . . . 3
1.3 Các thách thức khi tìm hiểu . . . 3

2 Giảm chiều dữ liệu: 3
2.1 Chiều dữ liệu là gì?: . . . 3
2.2 Tại sao phải giảm chiều dữ liệu . . . 4
2.3 Giảm chiều dữ liệu là gì? . . . 5

3 PCA Algorithm: 5
3.1 Giới thiệu PCA: . . . 5
3.2 Cơ sở toán học: . . . 6
3.2.1 Kỳ vọng(Mean): . . . 6
3.2.2 Phương sai (variance): . . . 6
3.2.3 Hiệp phương sai (covariance): . . . 6
3.2.4 Ma trận hiệp phương sai: . . . 6
3.2.5 Hệ trực giao và trực chuẩn: . . . 7
3.2.6 Trị riêng (eigenvalue), vector riêng(eigenvector) của covariance matrix: . . . 8
3.3 Ý tưởng chính đằng sau PCA: . . . 9
3.4 Ý tưởng chính của PCA: . . . 10
3.4.1 Tại sao lại phải maximum variance? . . . 10
3.4.2 Tại sao lại phải tính toán PCs? . . . 10
3.5 Ưu và nhược điểm của PCA: . . . 10
3.5.1 Ưu điểm: . . . 10
3.5.2 Nhược điểm: . . . 11
3.6 PCA Algorithm: . . . 11
3.6.1 Các bước thực hiện PCA . . . 11
3.6.2 Tại sao khi chúng ta tính vector riêng, giá trị riêng ma trận hiệp phương sai của ma trận dữ liệu lại tìm được hệ trục tọa độ mới . . . 13
3.7 Siêu tham số trong PCA . . . 15
3.7.1 Các siêu tham số: . . . 15
3.7.2 Cách chọn n_components cho PCA: . . . 16

4 Thử nghiệm 17
4.1 Giới thiệu về bộ dữ liệu thử nghiệm . . . 17
4.1.1 Phân tích và trực quan hóa dữ liệu: . . . 19
4.1.2 Các thách thức từ bộ dữ liệu . . . 21
4.2 Các phương pháp xử lý mất cân bằng dữ liệu . . . 22
4.3 Các mô hình máy học (Machine Learning) được sử dụng để đánh giá . . . 22
4.3.1 Naive Bayes . . . 22
4.3.2 Logistic Regression . . . 23
4.3.3 SVM linear . . . 25
4.3.4 SVM Non Linear . . . 28
4.3.5 Decision Tree . . . 31

4.4 Các độ đo được sử dụng 31
4.4.1 Accuracy, Precision, Recall, F1 Score 31
4.5 Kết quả và đánh giá thực nghiệm 32
4.5.1 Với dữ liệu mất cân bằng 33
4.5.2 Với dữ liệu cân bằng bằng phương pháp downsampling 34
4.5.3 Với dữ liệu cân bằng bằng phương pháp upsampling . . 36

5 Kết luận: 38
5.1 Các mục tiêu đạt được: 38
5.2 Các kinh nghiệm đạt được 38

Tài liệu tham khảo 38

1 Giới thiệu đồ án:

1.1 Sơ lược về đồ án

Đồ án này hướng đến tìm ra ý nghĩa của việc Dimensionality Reduction (giảm chiều dữ liệu) nói chung và tìm hiểu phương pháp giảm chiều dữ liệu Principal Component Analysis (PCA) nói riêng.

1.2 Mục tiêu của đồ án

Để thực hiện được đồ án, chúng em đã đặt ra những mục tiêu như sau:

- Hiểu được ý nghĩa của việc giảm chiều dữ liệu.
- Hiểu và nắm rõ được ý tưởng, cách thức thực hiện và cơ sở toán học đằng sau PCA.
- Ứng dụng được PCA trong thực tế và các bài toán liên quan.

1.3 Các thách thức khi tìm hiểu

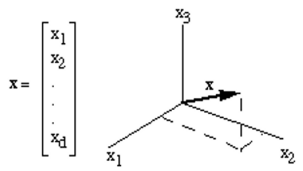
Khi chúng em thực hiện tìm hiểu đồ án, thì đã nhận ra được những khó khăn và thách thức như sau:

- Khó khăn về mặt kiến thức toán học và cơ sở toán học đằng sau PCA.
- Lúc mới bắt đầu tìm hiểu về PCA thì vẫn chưa định hình được PCA thực ra đang làm gì và tại sao nó lại phải làm như vậy.

2 Giảm chiều dữ liệu:

2.1 Chiều dữ liệu là gì?:

Dữ liệu hình ảnh(image data),Dữ liệu âm thanh, dữ liệu văn bản (text data)... đều được biểu diễn bởi các thuộc tính (không gian,thời gian. . .).Vậy dữ liệu k- chiều là dữ liệu được hoàn toàn xác định bởi vector các giá trị thuộc tính (x1,. . . xk) trong không gian k chiều.



Hình 1: Minh họa không gian vector trong không gian

2.2 Tại sao phải giảm chiều dữ liệu

Với thời đại dữ liệu bùng nổ như ngày nay, dữ liệu ta thu thập được rất lớn,Trong thực tế, các vector đặc trưng (feature vectors) có thể có số chiều rất lớn, tới vài nghìn. Đồng thời, lượng điểm dữ liệu cũng rất lớn ! Điều này sẽ gây khó khăn cho việc lưu trữ và tính toán, khó visualize không thể hiểu rõ về những feature mà chúng ta đang làm. Vì vậy, một trong những bước quan trọng trong nhiều bài toán ML,Data sience... là ta phải **giảm chiều dữ liệu** (dimentionality reduction).

Hình 2: Không gian dữ liệu nhiều chiều trong thực tế

2.3 Giảm chiều dữ liệu là gì?

Như đã đề cập, trong thực tế các feature vectors trong các bài toán thực tế có thể có số chiều rất lớn, tới vài nghìn. Ngoài ra, số lượng các điểm dữ liệu cũng thường rất lớn. Nếu thực hiện lưu trữ và tính toán trực tiếp trên dữ liệu có số chiều cao này thì sẽ gặp khó khăn cả về việc lưu trữ và tốc độ tính toán. Vì vậy, giảm số chiều dữ liệu là một bước quan trọng trong nhiều bài toán..
Vậy giảm chiều dữ liệu (dimensionality reduction, hay dimension reduction), là sự biến đổi dữ liệu từ không gian chiều cao thành không gian chiều thấp để biểu diễn ở dạng chiều thấp đồng thời giữ lại một số thuộc tính có ý nghĩa của dữ liệu gốc, có ý tưởng là gần với chiều nội tại (intrinsic dimension).Hay nói đơn giản là giảm chiều dữ liệ là việc đi tìm một hàm số, hàm số này lấy đầu vào là dữ liệu n-chiều rất lớn và tạo ra dữ liệu mới k-chiều (k<n).

3 PCA Algorithm:

3.1 Giới thiệu PCA:

Như đã đề cập ở phần giảm chiều dữ liệu, dữ liệu trong thực tế thực sự rất nhiều vầ rất lớn .Các vector đặc trưng (feature vectors) có thể có số chiều rất lớn, tới vài nghìn. Đồng thời, lượng điểm dữ liệu cũng rất lớn. Điều đó sẽ gây khó khăn cho việc lưu trữ và tính toán.Chúng ta đã đề cập 1 phương pháp rất chung là **giảm chiều dữ liệu** (dimentionality reduction)
Giảm chiều dữ liệu còn là phương pháp được sử dụng để giảm vấn đề quá khớp (overfitting),nó có hai hướng là hướng lựa chọn đặc trưng (feature selection) và hướng trích xuất đặc trưng (feature extraction). Với mục đích của đồ án, nhóm sẽ giới thiệu về một thuật toán theo hướng trích xuất đặc trưng là **Principal** Component Analysis (PCA), tức Phân tích thành phần chính. Phương pháp này dựa trên quan sát rằng dữ liệu thường không phân bố ngẫu nhiên trong không gian mà thường phân bố gần các đường/mặt đặc biệt nào đó. PCA xem xét một trường hợp đặc biệt khi các mặt đặc biệt đó có dạng tuyến tính là các không gian con (subspace)
Ngoài ra PCA còn được đánh giá là 1 phương pháp tuyệt vời dùng để giảm chiều dữ liệu, không chỉ vậy,mục tiêu của PCA còn mang đến nhiều đặc tính tốt khác như:

- PCA giúp giảm chiều dữ liệu rất nhiều, từ đó tối ưu không gian lưu trữ và những mục đích khác **(Nén ảnh,âm thanh...)**
- Thay vì giữ lại các trục tọa độ của không gian cũ, PCA xây dựng 1 không gian mới ít chiều hơn, nhưng lại có khả năng biểu diễn dữ liệu tốt tương đương không gian cũ.
- Trong không gian mới , các liên kết tiềm ẩn của dữ liệu có thể được khám phá, mà nếu đặt trong không gian cũ thì khó phát hiện hơn!
- PCA xây dựng 1 không gian mới ít chiều hơn, nhưng lại có khả năng biểu diễn dữ liệu tốt tương đương không gian cũ.
- Trích xuất được thành phần chính và loại bỏ thành phần dư thừa.

3.2 Cơ sở toán học:

Trước khi đi vào định nghĩa rõ ràng PCA, thì chúng ta sẽ cùng đề cập qua 1 số cơ sở toán học để xây dựng nên thuật toán này, để có thể có cái nhìn tổng quan và hiểu rõ hơn về nó.

3.2.1 Kỳ vọng(Mean):

Là giá trị mong muốn, nó đơn giản là trung bình cộng của toàn bộ các giá trị Cho N giá trị $x_1, x_2, \ldots, x_N$.

$$\bar{x} = \frac{1}{N}\sum_{i=1}^{N} x_i$$

3.2.2 Phương sai (variance):

Là trung bình cộng của bình phương khoảng cách từ mỗi điểm tới kỳ vọng, phương sai càng nhỏ thì các điểm dữ liệu càng gần với kỳ vọng, tức các điểm dữ liệu càng giống nhau. Phương sai càng lớn thì ta nói dữ liệu càng có tính phân tán.

$$\sigma^2 = \frac{1}{N-1}\sum_{i=1}^{N}(x_i - \bar{x})^2$$

3.2.3 Hiệp phương sai (covariance):

Là độ đo sự biến thiên cùng nhau của hai biến ngẫu nhiên (phân biệt với phương sai – đo mức độ biến thiên của một biến). Nếu 2 biến có xu hướng thay đổi cùng nhau (nghĩa là, khi một biến có giá trị cao hơn giá trị kỳ vọng thì biến kia có xu hướng cũng cao hơn giá trị kỳ vọng), thì hiệp phương sai giữa hai biến này có giá trị dương. Mặt khác, nếu một biến nằm trên giá trị kỳ vọng còn biến kia có xu hướng nằm dưới giá trị kỳ vọng, thì hiệp phương sai của hai biến này có giá trị âm. Nếu hai biến này độc lập với nhau thì giá trị bằng 0.

$$COV(X,Y) = \frac{\sum_{i=1}^{N}(X_i - \bar{X})(Y_i - \bar{Y})}{N}$$

3.2.4 Ma trận hiệp phương sai:

- Cho N điểm dữ liệu được biểu diễn bởi các vector cột $x_1, \ldots, x_N$, khi đó, vector kỳ vọng và ma trận hiệp phương sai của toàn bộ dữ liệu được định

nghĩa là:

$$\bar{x} = \frac{1}{N}\sum_{i=1}^{N} x_i$$

$$S = \frac{1}{N}\sum_{i=1}^{N}(x_i - \bar{x})(x_i - \bar{x})^T = \frac{1}{N}\hat{X}\hat{X}^T$$

- Ma trận hiệp phương sai là một ma trận đối xứng, hơn nữa, nó là một ma trận nửa xác định dương.
- Mọi phần tử trên đường chéo của ma trận hiệp phương sai là các số không âm. Chúng cũng chính là phương sai của từng chiều của dữ liệu.
- Các phần tử ngoài đường chéo $S_{ij}, i \neq j$ thể hiện sự tương quan giữa thành phần thứ i và thứ j của dữ liệu, còn được gọi là hiệp phương sai. Giá trị này có thể dương, âm hoặc bằng 0. Khi nó bằng 0, ta nói rằng hai thành phần i, j trong dữ liệu là không tương quan (uncorrelated).

- Nếu ma trận hiệp phương sai là ma trận đường chéo, ta có dữ liệu hoàn toàn không tương quan giữa các chiều.
  Minh hoạ ma trận hiệp phương sai:

$$S = \begin{bmatrix} var(x) & cov(x,y) \; cov(y,x) & var(y) \end{bmatrix}$$

3.2.5 Hệ trực giao và trực chuẩn:

Một hệ cơ sở $\mathbf{u}_1, \mathbf{u}_2, \ldots, \mathbf{u}_m \in \mathbb{R}^m$ được gọi là trực giao (orthogonal) nếu mỗi vector là khác 0 và tích của hai vector khác nhau bất kỳ bằng 0:

$$\mathbf{u}_i \neq \mathbf{0}; \;\; \mathbf{u}_i^T \mathbf{u}_j = 0 \; \forall \; 1 \leq i \neq j \leq m$$

Một hệ cơ sở $\mathbf{u}_1, \mathbf{u}_2, \ldots, \mathbf{u}_m \in \mathbb{R}^m$ được gọi là trực chuẩn (orthonormal) nếu nó là một hệ trực giao và độ dài Euclidean (norm 2) của mỗi vector bằng 1:

$$\mathbf{u}_i^T \mathbf{u}_j = \{1 \quad \text{if} \quad i = j \; 0 \quad \text{otherwise} \quad (1) \tag{1}$$

Gọi $\mathbf{U} = [\mathbf{u}_1, \mathbf{u}_2, \ldots, \mathbf{u}_m]$ là trực chuẩn, thế thì từ (1) có thể suy ra ngay:

$$\mathbf{U}\mathbf{U}^T = \mathbf{U}^T\mathbf{U} = \mathbf{I}$$

trong đó I là ma trận đơn vị bậc m. Ta gọi U là ma trận trực giao (orthogonal

matrix). Ma trận loại này không được gọi là ma trận trực chuẩn, không có định nghĩa cho ma trận trực chuẩn:
Một vài tính chất:

- $\mathbf{U}^{-1} = \mathbf{U}^T$: nghịch đảo của một ma trận trực giao chính là chuyển vị của nó.
- Nếu U là ma trận trực giao thì chuyển vị của nó $\mathbf{U}^T$ cũng là một ma trận trực giao.
- Định thức (determinant) của ma trận trực giao bằng 1 hoặc -1. Điều này có thể suy ra từ việc $\det(\mathbf{U})\det(\mathbf{U}^T) = \det(\mathbf{I}) = 1$.
- Ma trận trực giao thể hiện cho phép xoay (rotate) một vector. Giả sử có hai vector $\mathbf{x}, \mathbf{y} \in \mathbb{R}^m$ và ma trận trực giao $\mathbf{U} \in \mathbb{R}^{m\times m}$. Dùng ma trận này để xoay hai vector trên ta được $\mathbf{Ux}, \mathbf{Uy}$. Tích vô hướng của hai vector mới là:

  $$(\mathbf{Ux})^T(\mathbf{Uy}) = \mathbf{x}^T\mathbf{U}^T\mathbf{Uy} = \mathbf{x}^T\mathbf{y}$$

  như vậy phép xoay không làm thay đổi tích vô hướng giữa hai vector.
- Giả sử $\hat{\mathbf{U}} \in \mathbb{R}^{m\times r}, r < m$ là môt ma trận con của ma trận trực giao $\mathbf{U}$ được tạo bởir cột của U ta sẽ có $\hat{\mathbf{U}}^T\hat{\mathbf{U}} = \mathbf{I}_r$.

3.2.6 Trị riêng (eigenvalue), vector riêng(eigenvector) của covariance matrix:

Cho một ma trận vuông $A \in R^{n\times n}$,nếu số vô hướng λ và vector $x \neq 0 \in R^n$ thoả mãn:

$$Ax = \lambda x$$

thì λ được gọi là một trị riêng của A và x được gọi là vector riêng tương ứng với trị riêng đó.

- Trị riêng là nghiệm của phương trình đặc trưng

  $$det(A - \lambda I) = 0$$

- Một trị riêng có thể có nhiều vector riêng.
- Mỗi vector riêng chỉ ứng với một trị riêng duy nhất.
- Nếu x là một vector riêng của A ứng với λ thì $kx, k \neq 0$ cũng là vector riêng ứng với trị riêng đó.

- Mọi ma trận vuông bậc n đều có n trị riêng (kể cả lặp) và có thể là các số phức.

- Với ma trận đối xứng, tất cả các trị riêng đều là các số thực.

- Với ma trận xác định dương, tất cả các trị riêng của nó đều là các số thực dương. Với ma trận nửa xác định dương, tất cả các trị riêng của nó đều là các số thực không âm.

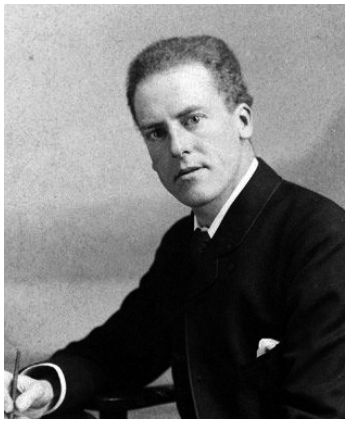
- Phương pháp giải tìm trị riêng, vector riêng:
  - B1: giải phương trình đặc trưng để tìm trị riêng:

$$det(A - \lambda I) = 0$$

  - B2: giải hệ phương trình tìm vector riêng u_i tương ứng với trị riêng λ_i.

3.3 Ý tưởng chính đằng sau PCA:

Như đã đề cập ở phần giới thiệu PCA, PCA là 1 thuật toán tuyệt vời trong việc giảm chiều dữ liệu. Nhưng ý tưởng đằng sau nó là gì? Và do đâu mà có sẽ được nhóm chúng em đề cập ở phần này. Đó chính là ý tưởng đằng sau PCA.
Ý tưởng ban đầu được phát hiện bởi tác giả Karl Pearson năm 1901 trong lúc ông đang nghiên cứu về di truyền. Sau này được ông Hotelling triển khai và mở rộng những thập niên 1930s.



Hình 3: Karl Pearson

Hình 4: Hotelling

Ý tưởng chính đằng sau PCA: " "**Describe the variation of multiple correlated variables by a group of new uncorrelated variables (components)**"
Có thể tạm hiểu là: nếu các biến số mà giữa chúng có liên quan đến nhau (multiple correlated variables) thì các các biến số này có thể giảm xuống thành

1(hoặc nhiều biến)(uncorrelated variables).Các biến này được gọi là **components**. Đây chính là ý tưởng tiền đề để có được PCA như ngày nay!

3.4 Ý tưởng chính của PCA:

Chúng ta vừa tìm hiểu về ý tưởng đằng sau PCA. Một ý tưởng rất quan trọng để tạo nên tiền đề phát triển nên PCA như ngày nay. Nhưng thực sự, điều PCA thực sự làm là gì ? Nó thực sự đã làm gì để có thể giảm chiều dữ liệu như vậy ? Tất cả sẽ được giải đáp ở phần này: **Ý tưởng chính của PCA**.
Ý tưởng chính của PCA:
Điều mà PCA cố gắng làm đó chính là giảm chiều từ N_d chiều xuống thành K_d chiều . Tức là thay vì giữ lại các trục tọa độ của không gian cũ, PCA xây dựng 1 không gian mới ít chiều hơn, nhưng lại có khả năng biểu diễn dữ liệu tốt tương đương không gian cũ. Sao cho độ biến thiên dữ liệu trên đó là lớn nhất có thể. PCA về cơ bản là một kỹ thuật giảm kích thước đơn giản, biến đổi các cột của bộ dữ liệu thành một tập các đặc trưng mới. Nó thực hiện điều này bằng cách tìm một tập hợp các hướng mới (như trục X và Y) giải thích sự biến đổi tối đa trong dữ liệu, tức là hướng đó ta tìm được **maximum của variance**.

3.4.1 Tại sao lại phải maximum variance?

Như đã đề cập ở phần toán học phương sai, variance thể hiện độ phân tán của dữ liệu, khi variance lớn thì độ phân tán lớn và ngược lại. Mà thuật toán là muốn lấy tối đa các thông tin, nên nó sẽ lấy theo hướng variance lớn, dữ liệu sẽ có độ biến thiên cao, mang lại nhiều thông tin.
Trục tọa độ hệ thống mới này được gọi là Principal Components (PCs). Các phép chiếu của dữ liệu gốc trên bộ trục tọa độ (PC) mới đóng vai trò là bộ dữ liệu được chuyển đổi mới.

3.4.2 Tại sao lại phải tính toán PCs?

Rất đơn giản, thông tin chứa trong một cột dữ liệu tỉ lệ thuận với lượng phương sai của nó. Tương tự, các PCs có lượng phương sai lớn sẽ mang lại nhiều thông tin và từ đó ta có thể chọn các PCs chứa nhiều thông tin nhất.

Tóm lại, điều PCA làm đó là tìm tập vector: 1 đường thẳng, 1 mặt phẳng, hay bất cứ gì có thể để dữ liệu chiếu lên đó. Sao cho độ biến thiên dữ liệu trên đó là lớn nhất có thể! Nhưng vấn đề đặt ra là, làm thế nào mà PCA có thể tìm ra được những tập vector này ? Nó sẽ được giải đáp trong phần tiếp theo.

3.5 Ưu và nhược điểm của PCA:

3.5.1 Ưu điểm:

- Là một thuật toán tuyệt vời trong việc nén và giảm chiều dữ liệu.
- Trong ML : việc giảm chiều dữ liệu khiến cho việc train model diễn ra hiệu quả hơn.

- Giúp ta loại bỏ những thành phần dữ liệu không quan trọng.

3.5.2 Nhược điểm:

- PCA sẽ hoạt động không thật sự tốt nếu dữ liệu của chúng ta phân tán.
- Mất mát dữ liệu.

3.6 PCA Algorithm:

Về mặt ý tưởng chính của PCA, cũng có thể hiểu là đơn giản như đã đề cập. Nhưng thật sự sâu bên trong nó hoạt động một cách như thế nào và tại sao lại làm như vậy chúng ta vẫn chưa nhắc đến. Phần này nhóm chúng em sẽ đi chi tiết vào thuật toán PCA.
Cách đơn giản nhất để giảm chiều dữ liệu từ D về $K < D$ là chỉ giữ lại K phần tử quan trọng nhất. Tuy nhiên, việc làm này chắc chắn chưa phải tốt nhất vì chúng ta chưa biết xác định thành phần nào là quan trọng hơn. Hoặc trong trường hợp xấu nhất, lượng thông tin mà mỗi thành phần mang là như nhau, bỏ đi thành phần nào cũng dẫn đến việc mất một lượng thông tin lớn.

Tuy nhiên, nếu chúng ta có thể biểu diễn các vector dữ liệu ban đầu trong một hệ cơ sở mới mà trong hệ cơ sở mới đó, tầm quan trọng giữa các thành phần là khác nhau rõ rệt, thì chúng ta có thể bỏ qua những thành phần ít quan trọng nhất.
Lấy một ví dụ về việc có hai camera đặt dùng để chụp một con người, một camera đặt phía trước người và một camera đặt trên đầu. Rõ ràng là hình ảnh thu được từ camera đặt phía trước người mang nhiều thông tin hơn so với hình ảnh nhìn từ phía trên đầu. Vì vậy, bức ảnh chụp từ phía trên đầu có thể được bỏ qua mà không có quá nhiều thông tin về hình dáng của người đó bị mất.
PCA chính là phương pháp đi tìm một hệ cơ sở mới sao cho thông tin của dữ liệu chủ yếu tập trung ở một vài toạ độ, phần còn lại chỉ mang một lượng nhỏ thông tin. Và để cho đơn giản trong tính toán, PCA sẽ tìm một hệ trực chuẩn để làm cơ sở mới.

3.6.1 Các bước thực hiện PCA

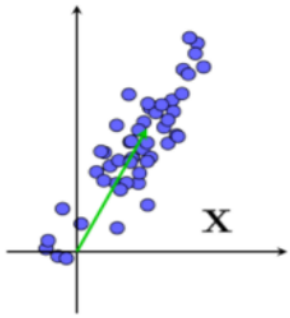
Từ các suy luận phía trên, ta có thể tóm tắt lại các bước trong PCA như sau:

- 1. Tính vector kỳ vọng của toàn bộ dữ liệu:

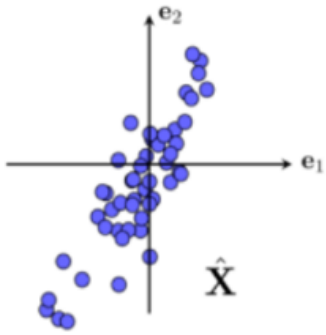
$$\bar{\mathbf{x}} = \frac{1}{N}\sum_{n=1}^{N}\mathbf{x}_n$$

- 2. Trừ mỗi điểm dữ liệu đi vector kỳ vọng của toàn bộ dữ liệu:

$$\hat{\mathbf{x}}_n = \mathbf{x}_n - \bar{\mathbf{x}}$$



Hình 5: 1. Tính vector kỳ vọng của toàn bộ dữ liệu

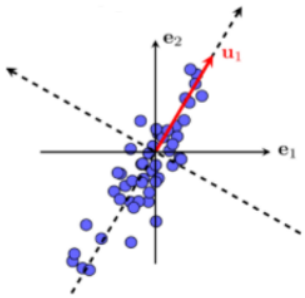


Hình 6: 2. Trừ mỗi điểm dữ liệu đi vector kỳ vọng của toàn bộ dữ liệu

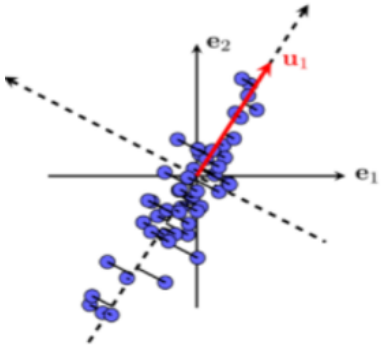
- 3.Tính ma trận hiệp phương sai:

$$\mathbf{S} = \frac{1}{N}\hat{\mathbf{X}}\hat{\mathbf{X}}^T$$

- 4. Tính các trị riêng và vector riêng có norm bằng 1 của ma trận này, sắp xếp chúng theo thứ tự giảm dần của trị riêng.
- 5. Chọn K vector riêng ứng với K trị riêng lớn nhất để xây dựng ma trận $\mathbf{U}_K$ có các cột tạo thành một hệ trực giao. K vectors này, còn được gọi là các thành phần chính, tạo thành một không gian con gần với phân bố của dữ liệu ban đầu đã chuẩn hoá.
- 6.Chiếu dữ liệu ban đầu đã chuẩn hoá $\hat{\mathbf{X}}$ xuống không gian con tìm được.



Hình 7: 5.Chọn K vector riêng



Hình 8: 6. Chiếu dữ liệu

- 7. Dữ liệu mới chính là toạ độ của các điểm dữ liệu trên không gian mới.

$$\mathbf{Z} = \mathbf{U}_K^T \hat{\mathbf{X}}$$

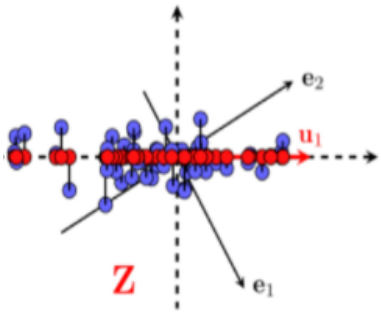
  xuống không gian con tìm được.

Dữ liệu ban đầu có thể tính được xấp xỉ theo dữ liệu mới như sau:

$$\mathbf{x} \approx \mathbf{U}_K \mathbf{Z} + \bar{\mathbf{x}}$$

3.6.2 Tại sao khi chúng ta tính vector riêng, giá trị riêng ma trận hiệp phương sai của ma trận dữ liệu lại tìm được hệ trục tọa độ mới

- **Tìm PC1:** giả sử chúng ta có ma trận dữ liệu X, chúng ta sẽ đi tìm thành phần chính thứ nhất (PC1) tương ứng việc đi tìm vector u_1 sao cho: phương sai của toàn bộ điểm dữ liệu X chiếu lên u_1 là lớn nhất và độ dài vector u_1 bằng 1 để đảm bảo không thay đổi cấu trúc phân bố dữ liệu. Và ta đại diện phát biểu trên theo toán học như sau:



Hình 9: 7. Dữ liệu mới

Tìm u_1 để

$$\begin{cases} \mathrm{u}_1 = argmax||{T_1}^{n*1}||_2{}^2 = argmax||X^{n*p}u_1||_2{}^2 \ (1) \\ ||\mathrm{u}_1||_2^2 = 1 \ (2) \end{cases}$$

Để tối ưu hàm số **(1)** theo ràng buộc **(2)**, ta sử dụng tối ưu ràng buộc Lagrange và đưa về được hàm số $L(u_1, \lambda)$:

$$L(u_1, \lambda_1) = ||X^{n*p}u_1||_2{}^2 - \lambda_1(||u_1||_2{}^2 - 1)$$

Như đã biết, để tối ưu hàm $L(u_1, \lambda)$ thì chúng ta sẽ đạo hàm, đặt chúng bằng 0 và giải phương trình.

$$\begin{cases} \mathrm{X}^T X u_1 = \lambda u_1 \\ ||\mathrm{u}_1||_2 = 1 \end{cases}$$

Vậy để tìm được u_1, ta sẽ tìm vector riêng của ma trận hiệp phương sai và giá trị càng lớn thì chứng tỏ vector riêng đó càng quan trọng.

- **Tìm PC2:** sau khi đã tìm được PC1, thì chúng ta sẽ tiếp tục đi tìm PC2. Tương tự như việc đi tìm PC1, nhưng cần thêm điều kiện là PC2 vuông góc với PC1. Biểu diễn phát biểu bằng toán học như sau:
  Tìm u_2 để

$$\begin{cases} \mathrm{u}_2 = argmax||{T_2}^{n*1}||_2{}^2 = argmax||X^{n*p}u_2||_2{}^2 \ (1) \\ ||\mathrm{u}_2||_2^2 = 1 \ (2) \\ \mathrm{u}_1.u_2 = 0 \ (3) \end{cases}$$

Chúng ta vẫn dùng tối ưu ràng buộc Lagrange,

$$L(u_2, \lambda_1, \lambda_2) = ||X^{n*p}u_2||_2{}^2 - \lambda_1(||u_1||_2{}^2 - 1) - \lambda_2 u_1 u_2)$$

Như đã biết, để tối ưu hàm $L(u_1, \lambda_1, \lambda_2)$ thì chúng ta sẽ đạo hàm, đặt chúng bằng 0 và giải phương trình.

$$\begin{cases} X^T X u_2 = \lambda u_2 \\ ||u_2||_2 = 1 \\ u_1 u_2 = 0 \end{cases}$$

Vậy để tìm được u_2, chúng ta vẫn sẽ tìm vector riêng của ma trận hiệp phương sai và giá trị càng lớn thì chứng tỏ vector riêng đó càng quan trọng.

- **Tương tự cho việc tìm PC thứ k**.

Kết luận: để tìm được hệ trục tọa độ mới để thỏa ý tưởng của PCA, ta sẽ thiết lập các điều kiện phương sai lớn nhất, độ dài vector bằng 1 và các vector cần phải vuông góc đôi một lẫn nhau. Khi đi tối ưu bằng Lagrange, để tìm được tập vector này thì phải đi tính ma trận hiệp phương sai, rồi tìm vector riêng, trị riêng của nó.

3.7 Siêu tham số trong PCA

3.7.1 Các siêu tham số:

Trong chương này, nhóm chúng tôi sẽ trình bày về các siêu tham số có trong thuật toán PCA. Theo như thư viện sklearn [1], PCA bao gồm các siêu tham số như sau:

- **n_components:** Số lượng thành phần cần giảm
- **copy:** Nếu giá trị bằng False, dữ liệu đầu vào sẽ bị ghi đè và được thực thi bằng lệnh *fit(X .transform (X))*, mục đích của việc này là khi thuật toán không mang lại kết quả như mong đợi.
- **whiten:** Nếu giá trị bằng True, các vectơ thành phần được nhân với căn bậc hai của n_components và sau đó chia cho các giá trị đơn lẻ để đảm bảo kết quả đầu ra không tương quan với các phương sai thành phần đơn vị. Việc sử dụng *whiten* sẽ loại bỏ một số thông tin khỏi dữ liệu được biến đổi (thang phương sai tương đối của các thành phần) nhưng đôi khi có thể cải thiện độ chính xác dự đoán của các bộ ước lượng bằng cách làm cho dữ liệu theo một số giả định phức tạp.
- **svd_solver:**
  - **auto:** Sử dụng bộ giải (solver) được chọn theo một cách mặc định dựa trên X.shape và n_components: nếu dữ liệu đầu vào lớn hơn 500x500 và số lượng thành phần cần trích xuất thấp hơn 80% kích thước nhỏ nhất của dữ liệu thì dữ liệu sẽ được xử lý bằng các phương pháp chọn ngẫu nhiên.

[1] https://scikit-learn.org/stable/modules/generated/sklearn.decomposition.PCA.html

- **full:** với tùy chọn này, thuật toán sẽ thực hiện SVD hoàn chỉnh với bộ giải LAPACK thông qua hàm *scipy.linalg.svd* và chọn các thành phần bằng cách hậu xử lý.
- **arpack:** với tùy chọn này, thuật toán sẽ thực hiện SVD được phân thành n_components với trình giải ARPACK thông qua hàm *scipy.sparse.linalg.svds*. Tùy chọn này yêu cầu giá trị nằm trong khoảng 0 < n_components < min(X.shape)
- **randomized:** chạy SVD ngẫu nhiên theo phương pháp của Halko et al.

- **tol:** Giá trị dung sai cho các giá trị đơn lẻ được tính toán khi svd_solver == 'arpack'. Giá trị nằm trong khoảng [0,0, vô cùng).
- **iterated_power:** Số lần lặp lại cho các phương pháp lũy thừa được tính toán khi svd_solver == 'randomizeda'. Giá trị nằm trong phạm vi [0, vô cùng).
- **random_state:** Được sử dụng khi bộ giải 'arpack' hoặc 'randomized' được sử dụng. Chuyển một số int cho các kết quả có thể lặp lại qua nhiều lệnh gọi hàm.

Tuy nhiên, đa phần việc chọn siêu tham số cho PCA đều tập trung vào giá trị n_components. Tác giả ở [3, 4] cũng đều tập trung vào việc điều chỉnh siêu tham số này để mô hình đạt được độ hiệu quả cao nhất. Vì vậy trong đồ án này, chúng tôi chỉ thực hiện việc tối ưu giá trị n_components.

3.7.2 Cách chọn n_components cho PCA:

Phương pháp 1: Việc chọn n_components hay nói cách khác là chọn k chiều của PCA là việc vô cùng quan trọng, tuy nhiên lại không có một phép tính hay công thức cụ thể nào tuyệt đối trong việc chọn k, nó tuỳ thuộc vào bài toán mà ta đang làm và kinh nghiệm của người cài đặt thuật toán. Việc chọn k sao cho hợp lý, trong đồ án này, nhóm chúng tôi sẽ đánh giá bằng công thức mục tiêu dưới đây.

$$\frac{\sum_{i=1}^{k} S_{ii}}{\sum_{i=1}^{m} S_{ii}} \geq 0.99$$

(99% of variance retained)

Hình 10: Công thức tính mức phương sai dữ liệu được giữ lại

Trong đó, S_{ii} đại diện cho trị riêng thứ i của ma trận trị riêng. Và những trị riêng này được sắp xếp theo thứ tự giảm dần. Trị riêng càng lớn, thì độ phân tán của những điểm dữ liệu chiếu lên vector riêng này càng lớn hoặc có thể nói lượng thông tin mà thành phần chính (PC) này đang lưu giữ càng nhiều. Công thức trên đang tính tỉ lệ giữa tổng k trị riêng đầu tiên và tổng của toàn bộ trị riêng tức là đang tính mức phương sai dữ liệu được giữ lại trên không gian mới k chiều.

Phương pháp 2: Trong các bài toán phân lớp hoặc hồi quy, chúng ta có thể sử dụng PCA nhằm giúp giảm chiều dữ liệu đầu vào cho các mô hình. Việc giảm chiều này giúp giảm overfitting khi dữ liệu ban đầu có quá nhiều trường dữ liệu dư thừa, cũng như tăng tốc độ tính toán cho các mô hình phân lớp, hồi quy. Tuy nhiên trong một số trường hợp, độ chính xác của các mô hình cũng sẽ giảm hoặc tăng.

Việc chọn siêu tham số n_component trong PCA là vô cùng quan trọng. Nếu chọn n_component càng thấp, thì số chiều dữ liệu giảm được càng nhiều, nhưng nó cũng sẽ đánh mất thông tin ban đầu càng nhiều dẫn đến độ chính xác của mô hình thấp. Nếu chọn n_component càng cao, thì lượng chiều dữ liệu giảm được càng thấp.

Độ lợi bộ nhớ là tỉ lệ giữa số lượng đặc trưng giảm được và số lượng đặc trưng ban đầu m:

$$C = \frac{m - n_component}{m}, C \in [0, 1]$$

, số lượng đặc trưng giảm được bằng số lượng đặc trưng ban đầu khi chưa được giảm chiều trừ cho số lượng đặc trưng còn lại sau khi được giảm chiều. Nếu số lượng đặc trưng còn lại sau khi giảm chiều càng ít, thì số lượng đặc trưng giảm được càng nhiều, dẫn lến độ lợi bộ nhớ C càng lớn và ngược lại.

Độ chính xác của mô hình phân lớp, hồi quy đạt được khi dùng PCA có n_component xác định là:

$$F = \text{Độ chính xác của mô hình}, F \in [0, 1]$$

Nhằm đưa ra một độ đo chung phục vụ việc chọn n_component một cách hợp lý, để sao cho độ lợi bộ nhớ C càng lớn và độ chính xác mô hình càng lớn, thì ta sử dụng harmonic-mean và trọng số ưu tiên để kết hợp 2 độ đo C và F:

$$H = \frac{(1 + \beta^2)FC}{\beta^2 F + C}, H \in [0, 1]$$

Trong một số trường hợp, ta có thể ưu tiên độ chính xác hơn là độ lợi về bộ nhớ hoặc ngược lại. Tham số β trên có thể giúp ta làm việc này.

- $\beta = 1$: ưu tiên giữa độ chính xác và lượng giảm chiều dữ liệu bằng nhau
- $\beta < 1$: ưu tiên độ chính xác cao hơn
- $\beta > 1$: ưu tiên độ lợi bộ cao hơn

4 Thử nghiệm

4.1 Giới thiệu về bộ dữ liệu thử nghiệm

Đồ án này hướng đến xây dựng một mô hình dự đoán những khách hàng có khả năng chấp nhận khoản vay từ ngân hàng bằng cách kết hợp thuật toán **PCA** và một số thuật toán Machine learning phân loại khác .Bộ Dataset được lấy từ **Universal Bank** [2] mà chúng tôi tham khảo từ Kaggle.

Một số trường thuộc tính và mô tả của bộ dữ liệu:

- ID: Customer ID.
- Age: Độ tuổi của khách hàng.
- Experience: Kinh nghiệm làm việc.
- Income: Thu nhập hằng tháng của khách hàng.
- ZIPCode: Mã ZIP .
- Family: Số thành viên trong gia đình của khách hàng
- CCAvg: Số tiền tiêu trung bình bằng thẻ tín dụng trên mỗi tháng.
- Education: Trinh độ học vấn:
  - Chưa tốt nghiệp
  - Đã tốt nghiệp
  - Thạc Sĩ/ Tiến Sĩ
- Mortage: Giá trị thế chấp nhà (nếu có)
- Personal Loan: Liệu khách hàng có chấp nhận khoảng vay cá nhân ? (Thuộc tính cần dự đoán)
- Securities Account: Liệu khách hàng có tài khoản an ninh của ngân hàng hay không ?
- CD Account: Liệu khách hàng có chứng chỉ tiền gửi của ngân hàng hay không ?
- Creditcard: Liệu khách hàng có sử dụng thẻ tín dụng của UniversalBank hay không ?

4.1.1 Phân tích và trực quan hóa dữ liệu:

Sau khi tiến hành đọc dữ liệu từ bộ dữ liệu được cung cấp, ta có bảng minh họa dữ liệu bên dưới:

ID	Age	Experience	Income	ZIP Code	Family	CCAvg	Education	Mortgage	Personal Loan	Securities Account	CD Account	Online	CreditCard
1	25	1	49	91107	4	1.6	1	0	0	1	0	0	0
2	45	19	34	90089	3	1.5	1	0	0	1	0	0	0
3	39	15	11	94720	1	1.0	1	0	0	0	0	0	0
4	35	9	100	94112	1	2.7	2	0	0	0	0	0	0
5	35	8	45	91330	4	1.0	2	0	0	0	0	0	1

Bảng 1: Minh họa dữ liệu

Tiến hành phân tích dữ liệu, nhóm nhận thấy bộ dữ liệu gồm **5000 điểm dữ liệu** và đúng **14 thuộc tính** như đã nêu ở trên, minh chứng ở bảng dưới đây:

	Unique Values
ID	5000
Age	45
Experience	47
Income	162
ZIP Code	467
Family	4
CCAvg	108
Education	3
Mortgage	347
Personal Loan	2
Securities Account	2
CD Account	2
Online	2
CreditCard	2

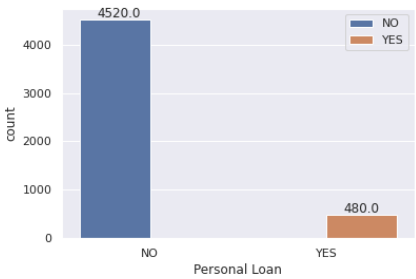
Hình 11: Bảng phân tích dữ liệu

Để kiểm tra tính đúng đắn của dữ liệu, nhóm đã tiến hành phân tích và kiểm thử trong bộ dữ liệu có bất kỳ trường nào bị lỗi hoặc trống hay không. Rất may mắn bộ dữ liệu hoàn toàn sạch và không có một trường dữ liệu nào bị lỗi, minh chứng ở bảng phía dưới đây:

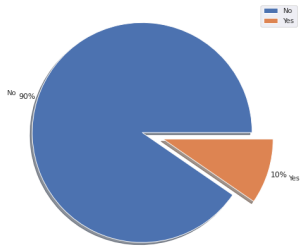
	Null Values
ID	0
Age	0
Experience	0
Income	0
ZIP Code	0
Family	0
CCAvg	0
Education	0
Mortgage	0
Personal Loan	0
Securities Account	0
CD Account	0
Online	0
CreditCard	0

Hình 12: Bảng phân tích dữ liệu có bị lỗi hay không

Trực quan hóa: Đối với đặc tính **Person Loan**, nhóm đã tiến hành trực quan hóa dữ liệu và nhận thấy rằng lớp dữ liệu 0 (Tương ứng với legend là No trong hình bên dưới), chiếm tới 90 % trong thuộc tính **Person Loan** và 10 % còn lại thuộc về lớp dữ liệu 1 (Tương ứng với legend là Yes).

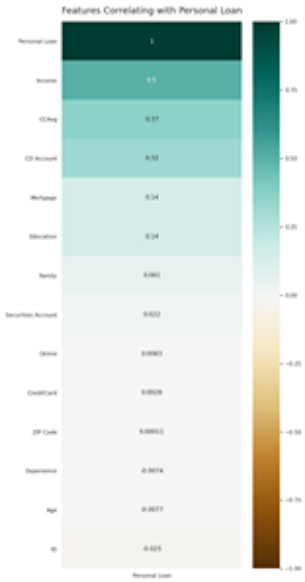


Hình 13: Barplot biểu diễn mức độ phân bố dữ liệu giữa các lớp



Hình 14: Piechart biểu diễn mức độ phấn bố dữ liệu giữa các lớp

Đối với 2 thuộc tính **Zipcode** và **ID** thì nhóm chung em quyết định loại bỏ chúng. Vì chúng em nhận thấy 2 thuộc tính này không ảnh hưởng đến kết quả của bài toán phân loại này. Đồng thời chúng em còn dựa vào bảng **feature correlation** [2] bên dưới để quyết định loại bỏ 2 thuộc tính này.



Hình 15: Zipcode và ID có giá trị thấp, ít ảnh hưởng tới thuộc tính cần dự đoán **Personal Loan**

Kết luận: Dựa vào việc trực quan hóa dữ liệu phía trên nhóm nhận thấy trong đặc tính **Person Loan** bị mất cân bằng rất lớn (class 0 chiếm tới 9 phần so với class 1), điều này có thể khiến cho độ chính xác cho các mô hình máy học có thể bị giảm sút so với kỳ vọng thực tế, chính vì thế nhóm đề xuất xử lý bài toán bằng các cách như sau:

- Chia class 0 ra thành 9 phần rồi lấy trung bình của nó, mục đích của việc này chính là để bộ dữ liệu được cân bằng so với class 1 chỉ chiếm 1 phần. Nhược điểm của cách này đó là làm cho bộ dữ liệu ít hơn cả ban đầu. Nhưng sau khi bàn bạc với các thành viên trong nhóm thì chúng em quyết định sử dụng đến phương án này.
- Nhóm còn sử dụng thêm các thuật toán và thử nghiệm với mục đích tìm ra phương pháp tăng độ chính xác tốt nhất như: PCA, ANN, Gradient Boosting, Decision Tree ,... Cụ thể hơn về các phương pháp này sẽ được nhóm trình bày phía bên dưới.
- Chúng em còn đề xuất sử dụng thêm các độ đo để phục vụ cho việc tính toán và đo đạc 1 cách hiệu quả như Ma trận nhầm lẫn, Precision, recall, F1 score.

4.1.2 Các thách thức từ bộ dữ liệu

Khi nhóm thực hiện phân tích và tìm hiểu bài toán thì nhóm đã nhận thấy được một số thách thức của bộ dữ liệu này như sau:

- Bộ dữ liệu tương đối ít, có thể khiến cho mô hình dễ bị quá khớp (Overfitting) dẫn đến kết quả không được như mong muốn.

- Đối với bộ dữ liệu được lựa chọn, sau khi tiến hành phân tích dữ liệu nhóm nhận thấy dữ liệu không được cân bằng, nên chúng em đã phải tiến hành các phương pháp khác nhau để có thể đạt được độ chính xác tối ưu nhất.

4.2 Các phương pháp xử lý mất cân bằng dữ liệu

- **Upsampling Data**: được sử dụng khi số lượng dữ liệu không đủ. Nó cố gắng cân bằng số liệu bằng cách tăng kích thước của các mẫu hiếm. Thay vì loại bỏ các mẫu phong phú, các mẫu hiếm mới được tạo ra bằng cách sử dụng ví dụ: lặp lại, bootstrapping hoặc SMOTE (Synthetic Minority Over-Sampling Technique) [1]. Lưu ý rằng không có lợi thế tuyệt đối của một phương pháp resampling hơn một phương pháp khác. Áp dụng hai phương thức này phụ thuộc vào trường hợp sử dụng mà nó áp dụng cho và tập dữ liệu chính nó.

- **Downsampling Data**: Việc lấy mẫu làm cân bằng tập dữ liệu bằng cách giảm kích thước của lớp trội. Phương pháp này được sử dụng khi số lượng dữ liệu là đủ. Bằng cách giữ tất cả các mẫu trong lớp hiếm và chọn ngẫu nhiên một số trong lớp trội, một tập dữ liệu cân bằng mới có thể được lấy ra để lập mô hình tiếp theo.

4.3 Các mô hình máy học (Machine Learning) được sủ dụng để đánh giá

4.3.1 Naive Bayes

Mô hình phân lớp là gì: Một mô hình phân lớp là một mô hình Machine Learning dùng để phân loại các vật mẫu dựa trên các đặc tính đã xác định.

Naive Bayes Clasifier Naive Bayes là một thuật toán phân lớp được mô hình hóa dựa trên định lý Bayes trong xác suất thống kê.
Xét bài toán classification với C classes $1, 2, \ldots, C$. Giả sử có một điểm dữ liệu $\mathbf{x} \in \mathbb{R}^d$. Ta cần tính:

$$p(y = c|\mathbf{x}) \quad (1)$$

Tức tính xác suất để đầu ra là class c biết rằng đầu vào là vector x.
Biểu thức này, nếu tính được, sẽ giúp chúng ta xác định được xác suất để điểm dữ liệu rơi vào mỗi class. Từ đó có thể giúp xác định class của điểm dữ liệu đó bằng cách chọn ra class có xác suất cao nhất:

$$c = \arg \max_{c \in \{1,\ldots,C\}} p(c|\mathbf{x}) \quad (2)$$

Hoặc chúng ta có thể tính theo biểu thức:

$$c = \arg\max_c p(c|\mathbf{x}) \quad (2)$$

$$= \arg\max_c \frac{p(\mathbf{x}|c)p(c)}{p(\mathbf{x})} \quad (3)$$

$$= \arg\max_c p(\mathbf{x}|c)p(c) \quad (4)$$

Thành phần còn lại $p(\mathbf{x}|c)$, tức phân phối của các điểm dữ liệu trong class c thường rất khó tính toán vì x là một biến ngẫu nhiên nhiều chiều, cần rất rất nhiều dữ liệu training để có thể xây dựng được phân phối đó. Để giúp cho việc tính toán được đơn giản, người ta thường giả sử một cách đơn giản nhất rằng các thành phần của biến ngẫu nhiên x là độc lập với nhau, nếu biết c. Tức là:

$$p(\mathbf{x}|c) = p(x_1, x_2, \ldots, x_d|c) = \prod_{i=1}^{d} p(x_i|c) \quad (6)$$

Giả thiết các chiều của dữ liệu độc lập với nhau, nếu biết c là quá chặt và ít khi tìm được dữ liệu mà các thành phần hoàn toàn độc lập với nhau. Tuy nhiên, giả thiết ngây ngô này lại mang lại những kết quả tốt bất ngờ. Giả thiết về sự độc lập của các chiều dữ liệu này được gọi là Naive Bayes (xin không dịch). Cách xác định class của dữ liệu dựa trên giả thiết này có tên là Naive Bayes Classifier (NBC).
NBC, nhờ vào tính đơn giản một cách ngây thơ, có tốc độ training và test rất nhanh. Việc này giúp nó mang lại hiệu quả cao trong các bài toán large-scale.
Ở bước training, các phân phối $p(c)$ và $p(x_i|c), i = 1, \ldots, d$ sẽ được xác định dựa vào training data. Việc xác định các giá trị này có thể dựa vào Maximum Likelihood Estimation hoặc Maximum A Posteriori. Ở bước test, với một điểm dữ liệu mới x class của nó sẽ được xác đinh bởi:

$$c = \arg\max_{c \in \{1,\ldots,C\}} p(c) \prod_{i=1}^{d} p(x_i|c) \quad (7)$$

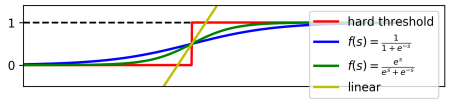
Khi d lớn và các xác suất nhỏ, biểu thức ở vế phải của (7) sẽ là một số rất nhỏ, khi tính toán có thể gặp sai số. Để giải quyết việc này (7) thường được viết lại dưới dạng tương đương bằng cách lấy log của vế phải:

$$c = \arg\max_{c \in \{1,\ldots,C\}} = \log(p(c)) + \sum_{i=1}^{d} \log(p(x_i|c)) \quad (7.1)$$

Mỗi giá trị $p(c), c = 1, 2, \ldots, C$ phụ thuộc vào loại dữ liệu.

4.3.2 Logistic Regression

Sơ lược: Logistic Regression là 1 thuật toán phân loại được dùng để gán các đối tượng cho 1 tập hợp giá trị rời rạc (như 0, 1, 2, ...). Một ví dụ điển hình là phân loại Email, gồm có email công việc, email gia đình, email spam, ... Giao dịch trực tuyến có là an toàn hay không an toàn, khối u lành tính hay ác tính. Thuật toán trên dùng hàm sigmoid logistic để đưa ra đánh giá theo xác suất.



Hình 16: Các activation function khác nhau.

Sigmoid function: Hàm Sigmoid là hàm có đầy đủ các tính chất sau:

- Là hàm số liên tục nhận giá trị thực, bị chặn trong khoảng (0,1)
- Nếu coi điểm có tung độ là 1/2 làm điểm phân chia thì các điểm càng xa điểm này về phía bên trái có giá trị càng gần 0. Ngược lại, các điểm càng xa điểm này về phía phải có giá trị càng gần 1. Điều này khớp với nhận xét rằng học càng nhiều thì xác suất đỗ càng cao và ngược lại.
- Mượt (smooth) nên có đạo hàm mọi nơi, có thể được lợi trong việc tối ưu.

Hàm Sigmoid:
Đây là hàm khá quan trọng trong thuật toán logistic regression. Nên chúng ta sẽ phải tìm hiểu nó trước khi đến với logistic sigmoid regression.

$$f(s) = \frac{1}{1+e^{-s}} \triangleq \sigma(s)$$

được sử dụng nhiều nhất, vì nó bị chặn trong khoảng (0,1).

$$\lim_{s\to-\infty} \sigma(s) = 0; \quad \lim_{s\to+\infty} \sigma(s) = 1$$

Thêm nữa:

$$\sigma'(s) = \frac{e^{-s}}{(1+e^{-s})^2} = \frac{1}{1+e^{-s}}\frac{e^{-s}}{1+e^{-s}} = \sigma(s)(1-\sigma(s))$$

Công thức cập nhật cho logistic sigmoid regression:

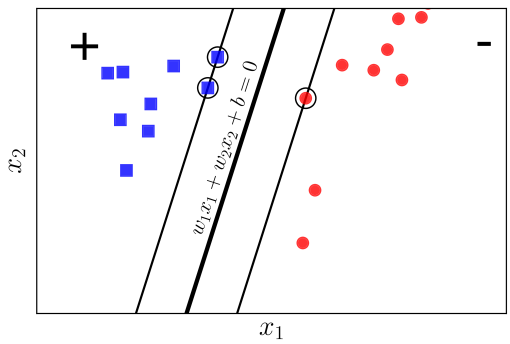
$$\frac{\partial J(\mathbf{w}; \mathbf{x}_i, y_i)}{\partial \mathbf{w}} = (z_i - y_i)\mathbf{x}_i$$

và công thức cập nhật (theo thuật toán SGD **[3]**) cho logistic regression là:

$$\mathbf{w} = \mathbf{w} + \eta(y_i - z_i)\mathbf{x}_i$$

4.3.3 SVM linear

Hard Margin SVM: Giả sử rằng các cặp dữ liệu của training set là $(\mathbf{x}_1, y_1)$, $(\mathbf{x}_2, y_2), \ldots, (\mathbf{x}_N, y_N)$ với vector $Xi \in Rd$ thể hiện đầu vào của 1 điểm dữ liệu Yi là nhãn của điểm dữ liệu đó. d là số chiều của dữ liệu và N là số điểm dữ liệu. Gỉa sử rằng nhãn của mỗi điểm dữ liệu được xác định bởi $(\mathbf{Y_i} = 1)$ (class 1) hoặc $(\mathbf{Y_i} = -1)$ (class 2) giống như trong PLA.



Hình 17: Phân tích bài toán SVM

Giả sử rằng các điểm vuông xanh thuộc class , các điểm tròn đỏ thuộc class -1 và mặt $\mathbf{w}^T\mathbf{x} + b = w_1x_1 + w_2x_2 + b = 0$ là mặt phân chia giữa hai classes (Hình 1). Hơn nữa, class 1 nằm về phía dương, class -1 nằm về phía âm của mặt phân chia. Nếu ngược lại, ta chỉ cần đổi dấu của w và b.
Ta quan sát thấy một điểm quan trọng sau đây: với cặp dữ liệu $(\mathbf{x}_n, y_n)$ bất kỳ, khoảng cách từ điểm đó tới mặt phân chia là:

$$\frac{y_n(\mathbf{w}^T\mathbf{x}_n + b)}{||\mathbf{w}||_2}$$

Điều này có thể dễ nhận thấy vì theo giả sử ở trên, y_n luôn cùng dấu với phía của x_n. Từ đó suy ra y_n cùng dấu với $(\mathbf{w}^T\mathbf{x}_n + b)$, và tử số luôn là 1 số không âm.
Với mặt phần chia như trên, margin được tính là khoảng cách gần nhất từ 1 điểm tới mặt đó (bất kể điểm nào trong hai classes):

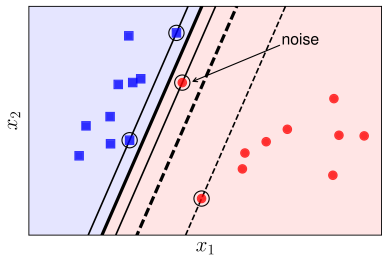
$$\text{margin} = \min_n \frac{y_n(\mathbf{w}^T\mathbf{x}_n + b)}{||\mathbf{w}||_2}$$

Bài toán tối ưu trong SVM chính là bài toán tìm w và b sao cho margin này đạt giá trị lớn nhất:

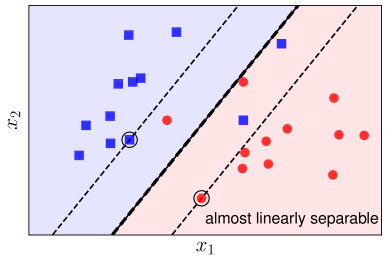
$$(\mathbf{w}, b) = \arg\max_{\mathbf{w},b} \left\{ \min_n \frac{y_n(\mathbf{w}^T\mathbf{x}_n + b)}{||\mathbf{w}||_2} \right\} = \arg\max_{\mathbf{w},b} \left\{ \frac{1}{||\mathbf{w}||_2} \min_n y_n(\mathbf{w}^T\mathbf{x}_n + b) \right\} \quad (1)$$

Soft Margin SVM:

Giới thiệu: Xét 2 ví dụ dưới đây:



Hình 18: a) SVM khi có nhiễu



Hình 19: b)dữ liệu gần linearly separable, SVM thuần không hoạt động hiệu quả.

Có hai trường hợp dễ nhận thấy SVM làm việc không hiệu quả hoặc thậm chí không làm việc:

- **Trường hợp 1**: Dữ liệu vẫn linearly separable như Hình 18a) nhưng có một điểm nhiễu của lớp tròn đỏ ở quá gần so với lớp vuông xanh. Trong trường hợp này, nếu ta sử dụng SVM thuần thì sẽ tạo ra một margin rất nhỏ. Ngoài ra, đường phân lớp nằm quá gần lớp vuông xanh và xa lớp tròn đỏ. Trong khi đó, nếu ta hy sinh điểm nhiễu này thì ta được một margin tốt hơn rất nhiều được mô tả bởi các đường nét đứt. SVM thuần vì vậy còn được coi là nhạy cảm với nhiễu (sensitive to noise).

- **Trường hợp 2**: Dữ liệu không linearly separable nhưng gần linearly separable như Hình 19b). Trong trường hợp này, nếu ta sử dụng SVM thuần thì rõ ràng bài toán tối ưu là infeasible, tức feasible set là một tập rỗng, vì vậy bài toán tối ưu SVM trở nên vô nghiệm. Tuy nhiên, nếu ta lại chịu hy sinh một chút những điểm ở gần biên giữa hai classes, ta vẫn có thể tạo được một đường phân chia khá tốt như đường nét đứt đậm. Các đường support đường nét đứt mảnh vẫn giúp tạo được một margin lớn cho bộ phân lớp này. Với mỗi điểm nằm lần sang phía bên kia của các đường suport (hay đường margin, hoặc đường biên) tương ứng, ta gọi điểm đó rơi vào vùng không an toàn. Chú ý rằng vùng an toàn của hai classes là khác nhau, giao nhau ở phần nằm giữa hai đường support.

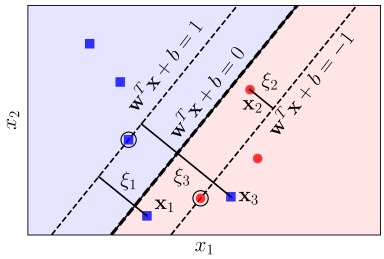
Trong cả hai trường hợp trên, margin tạo bởi đường phân chia và đường nét đứt mảnh còn được gọi là soft margin (biên mềm). Cũng theo cách gọi này, SVM thuần còn được gọi là Hard Margin SVM (SVM biên cứng).

Bản chất: **Soft Margin SVM** thực chất là 1 biến thể của **Hard Margin SVM** như đã đề cập ở trên.
Bài toán tối ưu cho Soft Margin SVM có hai cách tiếp cận khác nhau, cả hai đều mang lại những kết quả thú vị và có thể phát triển tiếp thành các thuật toán SVM phức tạp và hiệu quả hơn.

- **Cách 1:** giải một bài toán tối ưu có ràng buộc bằng cách giải bài toán đối ngẫu giống như Hard Margin SVM; cách giải dựa vào bài toán đối ngẫu này là cơ sở cho phương pháp Kernel SVM cho dữ liệu thực sự không linearly separable

- **Cách 2:** đưa về một bài toán tối ưu không ràng buộc. Bài toán này có thể giải bằng các phương pháp Gradient Descent

Phân tích toán học: Để có một margin lớn hơn trong Soft Margin SVM, chúng ta cần hy sinh một vài điểm dữ liệu bằng cách chấp nhận cho chúng rơi vào vùng không an toàn. Tất nhiên, chúng ta phải hạn chế sự hy sinh này, nếu không, chúng ta có thể tạo ra một biên cực lớn bằng cách hy sinh hầu hết các điểm. Vậy hàm mục tiêu nên là một sự kết hợp để tối đa margin và tối thiểu sự hy sinh.
Giống như với Hard Margin SVM, việc tối đa margin có thể đưa về việc tối thiểu $||\mathbf{w}||_2^2$



Hình 20: a) SVM khi có nhiễu

Với mỗi điểm $\mathbf{x}_n$trong tập toàn bộ dữ liệu huấn luyện, ta giới thiệu thêm một biến đo sự hy sinh ξ_n tương ứng. Biến này còn được gọi là slack variable. Với những điểm $\mathbf{x}_n$ nằm trong vùng an toàn, $\xi_n = 0$. Với mỗi điểm nằm trong vùng không an toàn như $\mathbf{x}_1, \mathbf{x}_2$, ta có $\xi_i > 0$.Nhận thấy rằng nếu $y_i = \pm 1$ là nhãn của $\mathbf{x}$ trong vùng không an toàn thì $\xi_i = |\mathbf{w}^T\mathbf{x}_i + b - y_i|$
Với Soft Margin SVM, hàm mục tiêu sẽ có thêm một số hạng nữa giúp tối thiểu sự hy sinh. Từ đó ta có hàm mục tiêu:

$$\frac{1}{2}||\mathbf{w}||_2^2 + C\sum_{n=1}^{N}\xi_n$$

Hằng số C được dùng để điều chỉnh tầm quan trọng giữa margin và sự hy sinh.
Ràng buộc mềm:

$$y_n(\mathbf{w}^T\mathbf{x}_n + b) \geq 1 - \xi_n \Leftrightarrow 1 - \xi_n - y_n(\mathbf{w}^T\mathbf{x}_n + b) \leq 0, \ \ \forall n = 1, 2, \ldots, n$$

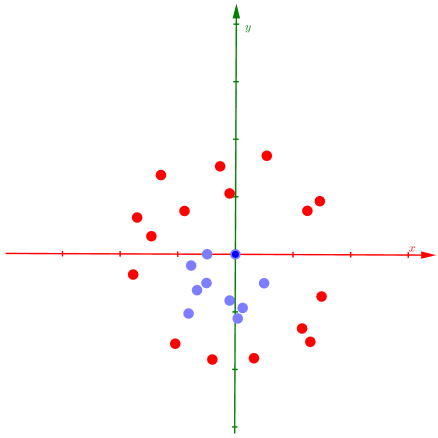
Và **ràng buộc phụ** $\xi_n \geq 0, \ \forall n = 1, 2, \ldots, N$
Tóm lại: ta có được hàm tối ưu ở dạng chuẩn cho Soft Margin SVM như sau:

$$(\mathbf{w}, b, \xi) \quad = \quad \arg\min_{\mathbf{w}, b, \xi} \frac{1}{2}||\mathbf{w}||_2^2 + C\sum_{n=1}^{N}\xi_n$$

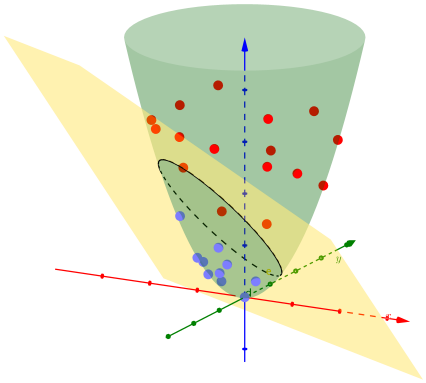
subject to: 1 - $\xi_n - y_n(\mathbf{w}^T\mathbf{x}_n + b) \leq 0, \forall n = 1, 2, \ldots, N$ (2) $-\xi_n \leq 0, \ \forall n = 1, 2, \ldots, N$(5)

4.3.4 SVM Non Linear

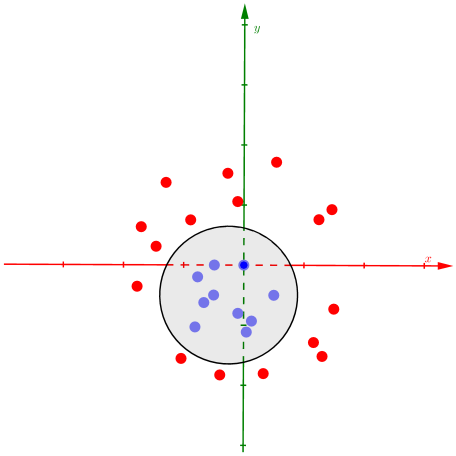
Sơ lược: Ý tưởng cơ bản của Kernel SVM và các phương pháp kernel nói chung là tìm một phép biến đổi sao cho dữ liệu ban đầu là không phân biệt tuyến tính được biến sang không gian mới. Ở không gian mới này, dữ liệu trở nên phân biệt tuyến tính.
Xét ví dụ dưới đây với việc biến dữ liệu không phân biệt tuyến tính trong không gian hai chiều thành phân biệt tuyến tính trong không gian ba chiều bằng cách giới thiệu thêm một chiều mới:



Hình 21: a)



Hình 22: b)



Hình 23: c)

Ví dụ về Kernel SVM:

- Dữ liệu của hai classes là không phân biệt tuyến tính trong không gian hai chiều.
- Nếu coi thêm chiều thứ ba là một hàm số của hai chiều còn lại $z = x^2 + y^2$,các điểm dữ liệu sẽ được phân bố trên 1 parabolic và đã trở nên phân biệt tuyến tính. Mặt phẳng màu vàng là mặt phân chia, có thể tìm được bởi Hard/Soft Margin SVM.
- Giao điểm của mặt phẳng tìm được và mặt parabolic là một đường ellipse, khi chiếu toàn bộ dữ liệu cũng như đường ellipse này xuống không gian hai chiều ban đầu, ta đã tìm được đường phân chia hai classses.
  Nói một cách ngắn gọn, Kernel SVM là việc đi tìm một hàm số biến đổi dữ liệu x từ không gian feature ban đầu thành dữ liệu trong một không gian mới bằng hàm số $\Phi(\mathbf{x})$

Một số hàm Kernel thông dụng:

Linear:
Như đã đề cập phía trên, đây là trường hợp đơn giản với kernel chính tích vô hướng của hai vector:

$$k(\mathbf{x}, \mathbf{z}) = (r + \gamma \mathbf{x}^T \mathbf{z})^d$$

Polynomial:
Với d là một số dương để chỉ bậc của đa thức. d có thể không là số tự nhiên vì mục đích chính của ta không phải là bậc của đa thức mà là cách tính kernel. Polynomial kernel có thể dùng để mô tả hầu hết các đa thức có bậc không vượt quá d. Nếu d là một số tự nhiên.

$$k(\mathbf{x}, \mathbf{z}) = \mathbf{x}^T \mathbf{z}$$

Radial Basic Function:
Radial Basic Function (RBF) kernel hay **Gaussian kernel** được sử dụng nhiều nhất trong thực tế, và là lựa chọn mặc định trong sklearn. Nó được định nghĩa bởi:

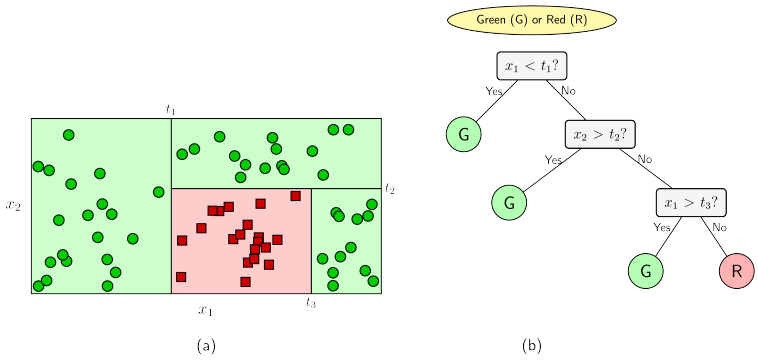
$$k(\mathbf{x}, \mathbf{z}) = \exp(-\gamma ||\mathbf{x} - \mathbf{z}||_2^2), \;\; \gamma > 0$$

Sigmoid:

$$k(\mathbf{x}, \mathbf{z}) = \tanh(\gamma \mathbf{x}^T \mathbf{z} + r)$$

4.3.5 Decision Tree

Decision Tree là một đồ thị của các quyết định và các hậu quả có thể của nó. Cây quyết định được sử dụng để xây dựng một kế hoạch nhằm đạt được mục tiêu mong muốn. Các cây quyết định được dùng để hỗ trợ quá trình ra quyết định. Cây quyết định là một dạng đặc biệt của cấu trúc cây.



Hình 24: Ví dụ về sử dụng Decision tree cho bài toán phân lớp

4.4 Các độ đo được sử dụng

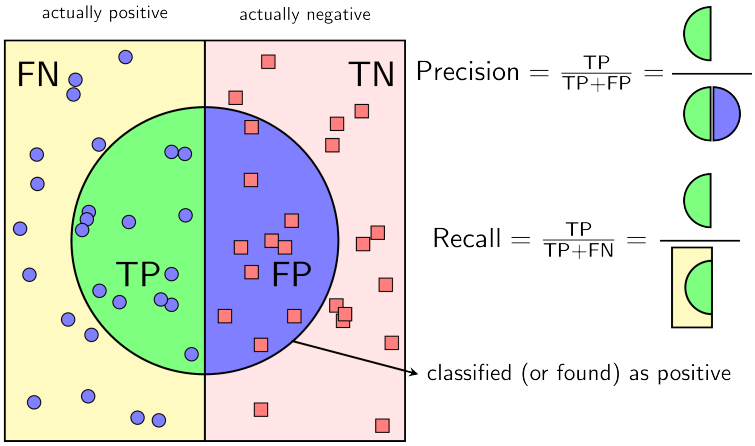
4.4.1 Accuracy, Precision, Recall, F1 Score

Accuracy: Cách đánh giá **Accuracy** đơn giản tính tỉ lệ giữa số điểm được dự đoán đúng và tổng số điểm trong tập dữ liệu kiểm thử.

$$\textit{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Hình 25: Công thức đánh giá Accuracy

Precision-Recall: Với bài toán phân loại mà tập dữ liệu của các lớp là chênh lệch nhau rất nhiều, có một phép đó hiệu quả thường được sử dụng là **Precision-Recall**.
Dưới đây là cách tính Precison-Recall đơn giản:



Hình 26: Minh hoạ và công thức đánh giá Precison-Recall

Khi Precision = 1, mọi điểm tìm được đều thực sự là positive, tức không có điểm negative nào lẫn vào kết quả. Tuy nhiên, Precision = 1 không đảm bảo mô hình là tốt, vì câu hỏi đặt ra là liệu mô hình đã tìm được tất cả các điểm positive hay chưa. Nếu một mô hình chỉ tìm được đúng một điểm positive mà nó chắc chắn nhất thì ta không thể gọi nó là một mô hình tốt.
Khi Recall = 1, mọi điểm positive đều được tìm thấy. Tuy nhiên, đại lượng này lại không đo liệu có bao nhiêu điểm negative bị lẫn trong đó. Nếu mô hình phân loại mọi điểm là positive thì chắc chắn Recall = 1, tuy nhiên dễ nhận ra đây là một mô hình cực tồi.
Một mô hình phân lớp tốt là mô hình có cả Precision và Recall đều cao, tức càng gần một càng tốt.

F1-Score: F1-score, là harmonic mean của precision và recall (giả sử rằng hai đại lượng này khác không):

$$\frac{2}{F_1} = \frac{1}{\text{precision}} + \frac{1}{\text{recall}} \text{ hay } F_1 = 2\frac{1}{\frac{1}{\text{precision}} + \frac{1}{\text{recall}}} = 2\frac{\text{precion} \cdot \textit{recall}}{\text{precision} + \text{recall}}$$

F1 Score có giá trị nằm trong nửa khoảng (0,1]. F_1 score càng cao, bộ phân lớp càng tốt. Khi cả recall và precision đều bằng 1 (tốt nhất có thể), $F_1 score = 1$ Khi cả recall và precision đều thấp.

4.5 Kết quả và đánh giá thực nghiệm

Như đã đề cập ở 4.1.1, bộ dữ liệu mà chúng tôi thực hiện bao gồm 5000 điểm dữ liệu không bao gồm tập test. Vì vậy, để xác nhận độ hiệu quả của mô hình, chúng tôi tiếp hành chia tập dữ liệu thành hai tập huấn luyện và kiểm thử với tỉ lệ 8:2.

4.5.1 Với dữ liệu mất cân bằng

Trong thí nghiệm này, chúng tôi tiến hành đánh giá việc giữ nguyên việc mất cân bằng dữ liệu và thực hiện huấn luyện các mô hình máy học với hai trường hợp áp dụng PCA và không áp dụng PCA.

1. **Dùng PCA:**
   Với lần thực nghiệm này, nhóm chúng tôi tiến hành huấn luyện các mô hình máy học trên bộ dữ liệu bị mất cân bằng về nhãn và áp dụng mô hình PCA, các mô hình được cài đặt với các siêu tham số theo mặc định.

   Đầu tiên, chúng tôi đi tìm số k chiều cần giảm sao cho tối ưu về tính chính xác cũng như tốc độ xử lý với cách tính đã đề cập ở mục 3.7.2 với kết quả tìm được ở bảng 2 như sau:

Bảng 2: Bảng kết quả việc chọn siêu tham số n_components cho từng mô hình máy học

Name Algo	n_component	harmonic_mean
Decision Tree	1	0.956522
Gaussian Naive Bayes	2	0.721491
Support Vector Machine	3	0.708593
Logistic Regression	2	0.692811

   Với số n_components chọn được từ bảng 2, nhóm tiến hành huấn luyện các mô hình và thu được kết quả ở bảng 3 sau:

Bảng 3: Bảng kết quả trên dữ liệu mất cân bằng áp dụng PCA

ML Algorithm	Accuracy	Precision	Recall	F1-score
Gaussian Naive Bayes	0.874	0.682522	**0.634940**	**0.653080**
Support Vector Machine	**0.902**	**0.858729**	0.606768	0.646847
Logistic Regression	0.890	0.744398	0.607306	0.638686
Decision Tree	0.829	0.556935	0.547033	0.550746

   Trong bảng 3, các kết quả từ các độ đo như Precision, Recall và F1-score không quá chênh lệch nhau. Lần thực nghiệm này cho kết quả cao nhất với Gaussian Naive Bayes cho điểm số F1-score cao nhất là **0.653** và Decision Tree cho kết quả F1-score thấp nhất là **0.550**.

2. **Không dùng PCA:**
   Với lần thực nghiệm này, nhóm chúng tôi tiến hành huấn luyện các mô hình máy học trên bộ dữ liệu bị mất cân bằng về nhãn và không áp dụng mô hình PCA, các mô hình được cài đặt với các siêu tham số theo mặc định. Kết quả thu được ở bảng 4 như sau:

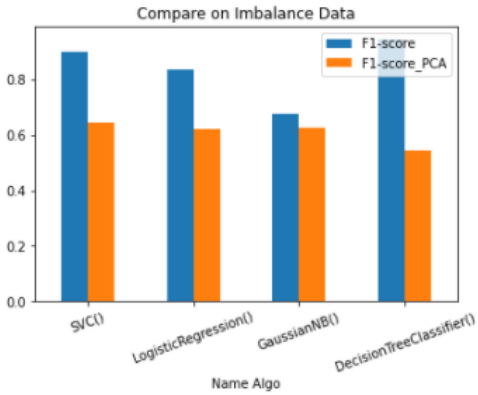
   Kết quả từ bảng 4 cho thấy rằng, tuy đánh giá trên bộ dữ liệu mất cân bằng nhưng các mô hình máy học mà chúng tôi áp dụng đều cho kết quả rất khả quan. Với mô hình Decision Tree cho kết quả cao nhấ,t lần lượt với Accuracy là **0.985**, Recall là **0.491** và F1-score là **0.9515**. Mặc khác, các thuật toán còn lại cũng cho kết quả tương đối. Duy chỉ với Gaussian

Bảng 4: Bảng kết quả trên dữ liệu mất cân bằng không áp dụng PCA

ML Algorithm	Accuracy	Precision	Recall	F1-score
Decision Tree	**0.985**	0.953967	**0.949116**	**0.951525**
Support Vector Machine	0.984	**0.978750**	0.916554	0.945024
Logistic Regression	0.955	0.870016	0.826005	0.846269
Gaussian Naive Bayes	0.870	0.646997	0.726197	0.673222

Naive Bayes thì kết quả lại thấp hơn nhiều, tuy rằng accuracy nằm ở mức **0.870** nhưng các độ đo như Precision hay Recall đều nằm ở mức 0.6-0.7.

3. **So sánh kết quả giữa hai phương pháp:**



Hình 27: So sánh số điểm F1-score giữa việc áp dụng và không áp dụng PCA

Hình 27 so sánh điểm F1-score giữa hai việc áp dụng PCA và không áp dụng PCA cho dữ liệu bị mất cân bằng. Ta nhận thấy rằng việc không áp dụng PCA mang lại kết quả cao hơn rất nhiều so với việc áp dụng PCA.

4.5.2 Với dữ liệu cân bằng bằng phương pháp downsampling

Trái ngược với lần trước, ở thí nghiệm này chúng tôi tiến hành thực nghiệm trên tập dữ liệu đã được cân bằng bằng phương pháp **downsampling** như đã giới thiệu ở 4.2 và thực hiện huấn luyện các mô hình máy học với hai trường hợp áp dụng PCA và không áp dụng PCA.

1. Dùng PCA

Bảng 5: Bảng kết quả việc chọn siêu tham số n_components cho từng mô hình máy học

Name Algo	**n_component**	**harmonic_mean**
Decision Tree	1	0.956522
Gaussian Naive Bayes	2	0.728374
Support Vector Machine	3	0.710296
Logistic Regression	2	0.707925

Với số n_components chọn được từ bảng 5, nhóm tiến hành huấn luyện các mô hình và thu được kết quả ở bảng 6 sau:

Bảng 6: Bảng kết quả trên dữ liệu cân bằng áp dụng PCA

ML Algorithm	**Accuracy**	**Precision**	**Recall**	**F1-score**
Decision Tree	0.714777	0.580535	0.727376	0.563628
Support Vector Machine	0.793637	0.636652	**0.852717**	0.653146
Logistic Regression	0.820980	0.633604	0.799730	0.658992
Gaussian Naive Bayes	**0.852070**	**0.643838**	0.766507	**0.674086**

Trong bảng 6 các kết quả từ các độ đo như Precision, Recall và F1-score không quá chênh lệch nhau. Lần thực nghiệm này cho kết quả cao nhất vẫn là Gaussian Naive Bayes cho điểm số F1-score cao nhất là **0.674086** và Decision Tree cho kết quả F1-score thấp nhất là **0.563628**.

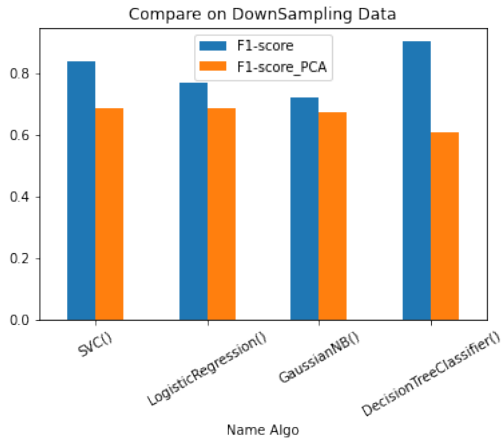
2. Không dùng PCA
   Với lần thực nghiệm này, nhóm chúng tôi tiến hành huấn luyện các mô hình không áp dụng mô hình PCA, các mô hình được cài đặt với các siêu tham số theo mặc định. Kết quả thu được ở bảng 7 như sau:

Bảng 7: Bảng kết quả trên dữ liệu cân bằng không áp dụng PCA

ML Algorithm	**Accuracy**	**Precision**	**Recall**	**F1-score**
Decision Tree	**0.958687**	0.841381	**0.969108**	**0.890774**
Support Vector Machine	0.910238	**0.742712**	0.942925	0.799752
Logistic Regression	0.883379	0.708053	0.918660	0.758244
Gaussian Naive Bayes	0.842496	0.652296	0.822680	0.685387

Kết quả từ bảng 7 cho thấy rằng, đánh giá trên bộ dữ liệu cân bằng được sử dụng **downsampling** thì kết quả đã có sự thay đổi đáng kể là giá trị của F1-score đã giảm xuống so với bảng 4. Mô hình Decision Tree vẫn cho kết quả cao nhất lần lượt với Accuracy là **0.958687**, Recall là **0.969108** và F1-score là **0.890774**. Mặt khác, các thuật toán còn lại cũng cho kết quả tương đối. Duy chỉ với Gaussian Naive Bayes thì kết quả F1-score lại thấp hơn nhiều vì thuật toán này cho ra kết quả Precision và Recall khá lệch nhau.

3. **So sánh kết quả giữa hai phương pháp:**



Hình 28: So sánh số điểm F1-score giữa việc áp dụng và không áp dụng PCA

Hình 28 so sánh điểm F1-score giữa hai việc áp dụng PCA và không áp dụng PCA cho dữ liệu đã được cân bằng với phương pháp **downsampling**. Ta nhận thấy rằng việc không áp dụng PCA mang lại kết quả vẫn cao hơn so với việc áp dụng PCA. Tuy nhiên nếu so biểu đồ ở hình 27 thì sự chênh lệch giữa việc dùng so với không dùng PCA đã rút ngắn hơn. Đặc biệt là với thuật toán **Gaussian Naive Bayes** thì có thể xem như điểm F1-score đã ngang bằng nhau ở cả 2 phương pháp. Điều này chứng tỏ việc cân bằng dữ liệu không những cải thiện kết quả bài toán mà còn đảm bảo một độ chính xác chấp nhận được khi sử dụng PCA để tối ưu không gian lưu trữ.

4.5.3 Với dữ liệu cân bằng bằng phương pháp upsampling

Không chỉ áp dụng một phương pháp để cân bằng dữ liệu là **downsampling**, ở thí nghiệm này nhóm chúng tôi còn thực hiện thêm phương pháp **upsampling** để có kết quả đánh giá và so sánh khách quan nhất.

1. Dùng PCA

Bảng 8: Bảng kết quả việc chọn siêu tham số n_components cho từng mô hình máy học

Name Algo	n_component	harmonic_mean
Decision Tree	1	0.956522
Gaussian Naive Bayes	2	0.728374
Support Vector Machine	3	0.710296
Logistic Regression	2	0.707925

Với số n_components chọn được từ bảng 8 , nhóm tiến hành huấn luyện các mô hình và thu được kết quả ở bảng 9 sau:

Bảng 9: Bảng kết quả trên dữ liệu cân bằng áp dụng PCA

ML Algorithm	**Accuracy**	**Precision**	**Recall**	**F1-score**
Decision Tree	0.760	0.601659	0.767470	0.603709
Support Vector Machine	0.801	0.643977	**0.869913**	0.663669
Logistic Regression	0.837	**0.653807**	0.841562	**0.686388**
Gaussian Naive Bayes	**0.850**	0.648341	0.789971	0.680862

Trong bảng 9 các kết quả từ các độ đo như Precision, Recall và F1-score không quá chênh lệch nhau. Lần thực nghiệm mô hình cho kết quả F1-score cao nhất là **Logistic Regression** với giá trị là **0.686388**. Hơn nữa, nếu so sánh với kết quả tương tự ở bảng 6 thì rõ ràng phương pháp **upsampling** được chúng tôi sử dụng đã mang lại kết quả nhỉnh hơn một tí so với **downsampling**.

2. Không dùng PCA
   Với lần thực nghiệm này, nhóm chúng tôi tiến hành huấn luyện các mô hình không áp dụng mô hình PCA, các mô hình được cài đặt với các siêu tham số theo mặc định. Kết quả thu được ở bảng 10 như sau:
   Kết quả từ bảng 10 cho thấy đánh giá trên bộ dữ liệu cân bằng được

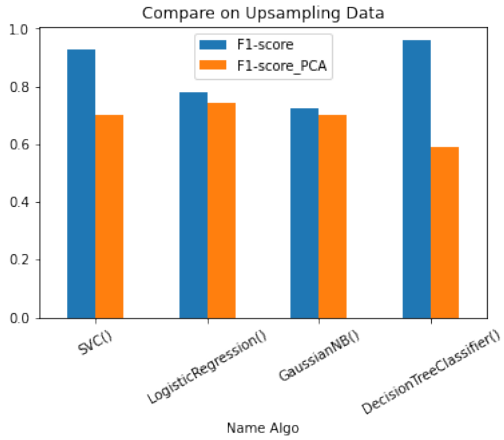
Bảng 10: Bảng kết quả trên dữ liệu cân bằng không áp dụng PCA

ML Algorithm	**Accuracy**	**Precision**	**Recall**	**F1-score**
Decision Tree	**0.982**	**0.921019**	**0.974156**	**0.945599**
Support Vector Machine	0.971	0.877948	0.968145	0.916846
Logistic Regression	0.892	0.716465	0.919640	0.768786
Gaussian Naive Bayes	0.852	0.663004	0.839087	0.699831

sử dụng phương pháp **upsampling** có kết quả cao hơn so với phương pháp **downsampling**. Mô hình Decision Tree lần này còn chiếm ưu thế hoàn toàn trên toàn bộ các chỉ số được dùng để đánh giá. Duy chỉ có mô hình Gaussian Naive Bayes vẫn mang một kết quả thấp nhất trong các mô hình.

3. **So sánh kết quả giữa hai phương pháp:**

   Hình 29 so sánh điểm F1-score giữa hai việc áp dụng PCA và không áp dụng PCA cho dữ liệu đã được cân bằng với phương pháp **upsampling**. Ta nhận thấy rằng việc không áp dụng PCA mang lại kết quả vẫn cao hơn so với việc áp dụng PCA. Nhìn chung thì kết quả cũng khá tương đồng với bảng **downsampling** 28, tuy nhiên kết quả ở phương pháp **downsampling** lại có sự chênh lệch giữa việc dùng PCA và không dùng PCA là ít hơn so với phương pháp **upsampling**.



Hình 29: So sánh số điểm F1-score giữa việc áp dụng và không áp dụng PCA

5 Kết luận:

5.1 Các mục tiêu đạt được:

- Năm được các kiến thức quan trọng về thuật toán PCA và có thể code được 1 thuật toán PCA hoàn chỉnh.
- Hiểu được cách tinh chỉnh các siêu tham số sao cho phù hợp với bài toán.
- Hiểu được tầm quan trọng của việc giảm chiều dữ liệu.

5.2 Các kinh nghiệm đạt được

- Biết cách tổ chức và làm việc nhóm để công việc đạt hiệu quả tốt nhất.
- Tăng thêm khả năng lập trình. Cụ thể có khả năng xử lí các phép biến đổi trên các loại dữ liệu đa phương tiện.

Tài liệu tham khảo

[1] Kevin W. Bowyer et al. "SMOTE: Synthetic Minority Over-sampling Technique". In: *CoRR* abs/1106.1813 (2011). arXiv: 1106.1813. URL: http://arxiv.org/abs/1106.1813.

[2] *Dataset: Universal Bank*. URL: https://www.kaggle.com/kartikmohan1999/universal-bank/data.

[3] Aurlien Gron. *Hands-On Machine Learning with Scikit-Learn and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems*. 1st. O'Reilly Media, Inc., 2017. ISBN: 1491962291.

[4] Fabian Pedregosa et al. "Scikit-Learn: Machine Learning in Python". In: *J. Mach. Learn. Res.* 12.null (Nov. 2011), pp. 2825–2830. ISSN: 1532-4435.